

(SLL-Questions)

(Question # 01)

Void ReverseList() :

```
{  
    if(head == NULL || head->next == NULL)  
    {  
        return;  
    }  
}
```

// return if no node or only 1 node as
no need to reverse it

Now // if more than 1 node

```
Node* prev = NULL;    tail = head; // point  
Node* curr = head;    to head  
Node* nextn = NULL;
```

// we will use these pointers for shifting
nodes.

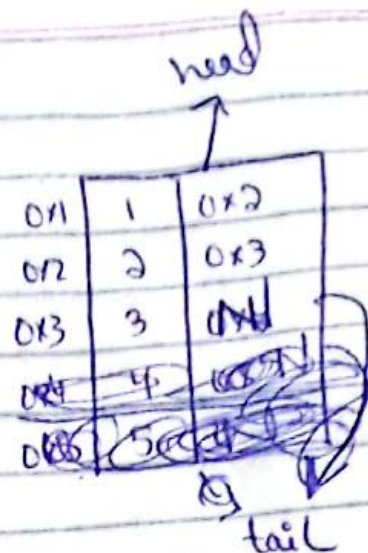
```
while(curr != NULL) // run loop until curr = NULL  
{
```

```
    nextn = curr->next; // move curr's next into nextn  
    curr->next = prev; // move prev into curr's next  
    prev = curr; // point prev to curr  
    curr = nextn; // point/assign next to curr  
}
```

```
    head = prev; // at the end point head to prev  
}
```

(DRY RUN)

Suppose Our Linked List is



Now:

Before iterations:

tail $\rightarrow$ 0x1
 prev = NULL
 curr $\rightarrow$ 0x1 (head)
 next = NULL

Now:

(curr $\neq$ NULL) ✓

```
{
    next = curr -> next;
    curr -> next = prev;
    prev = curr;
    curr = next;
}
```

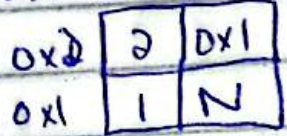
next = 0x2
 0x1 $\rightarrow$ next = NULL
 prev = 0x1
 curr = 0x2



again:

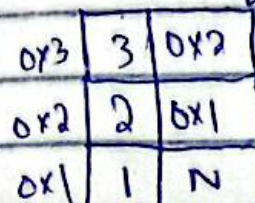
```
while (curr  $\neq$  NULL) ✓
{
    next = curr -> next;
    curr -> next = prev;
    prev = curr;
    curr = next;
}
```

next = 0x3
 0x2 $\rightarrow$ next = 0x1
 prev = 0x2
 curr = 0x3



again:

```
while ( ) ✓
{
    next = curr -> next;
    curr -> next = prev;
    prev = curr;
    curr = next;
}
```



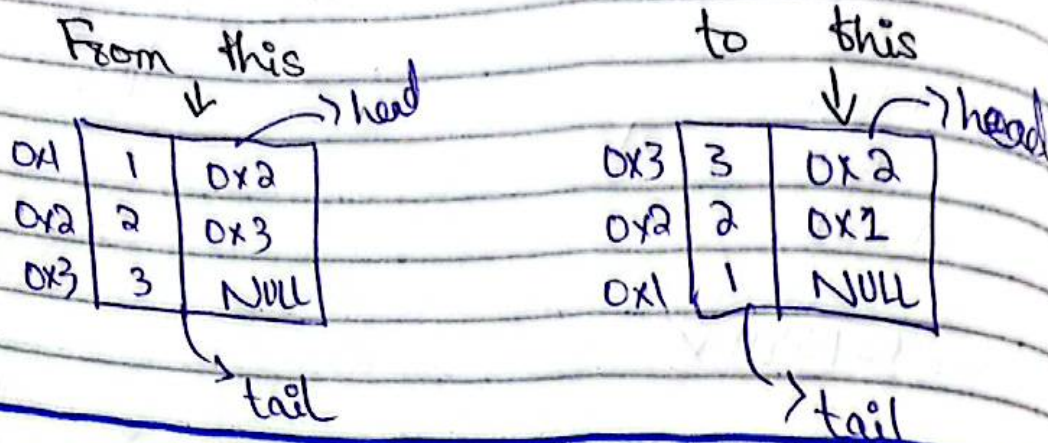
next = ~~0x4~~
 0x3 $\rightarrow$ next = 0x2
 prev = 0x3
 curr = ~~0x4~~

```

head = prev;
while (curr != NULL) X
{
    head = prev; // point head to prev
}

```

So our list became:



(Question # 02)

```

void merge(SLL &L2)
{

```

```

    Node* h1 = head; // head of L1
    Node* h2 = L2.head;

```

```

    if (h1 == NULL) // if L1 is empty
    {

```

```

        head = h2;
        tail = L2.tail;
        return;
    }

```

```

    if (h2 == NULL) // if L2 is empty
    {
        return;
    }
}

```

Node* newhead = NULL; // for new merged list
Node* newtail = NULL;

if (h1->data <= h2->data) // for sorted list
{
 newhead = newtail = h1; // point both to h1;
 h1 = h1->next; // to save head
}
else (h2->data < h1->data) // for initial head & tail }
{
 newhead = newtail = h2;
 h2 = h2->next;
}

while (h1 != NULL && h2 != NULL) // run until any head = NULL

{
 if (h1->data <= h2->data) // ~~if~~
 {
 newtail->next = h1; // link to newtail
 newtail = h1; // update newtail
 h1 = h1->next; // iterate h1
 }

else (h2->data < h1->data)

{
 newtail->next = h2; // Same
 newtail = h2; // Same
 h2 = h2->next; // iterate h2
}

if (h1 != NULL)

{
 newtail->next = h1;

if (h2 != NULL)

{
 newtail->next = h2;

// in case if loop finishes
 // bcz of any h1 or h2 so
 // link the remainings of any
 // to next of newtail

```

while(newtail->next != NULL) // if move tail to
{
    newtail = newtail->next;
}

head = newhead;
tail = newtail;
}

```

// update head & tail
new of merged

(DRY RUN)

Suppose: our 2 lists are:

0x1	1	0x3
0x3	3	0x5
0x5	5	N

0x2	2	0x4
0x4	4	0x5
0x5	5	N

→ L2

h1 = head

h1 → 0x1

h2 = h2.head

h2 → 0x2

newhead = NULL;

newtail = NULL

as $h1 < h2$ so

newhead = newtail = 0x1

Now:

h1 = h1 → next;

h1 = 0x3

Now: while (h1 != N && h2 != N) ✓

{ if (h1->d <= h2->d) X 3 < 2

{ - }

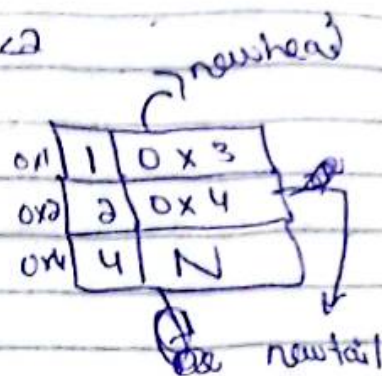
else

{ newtail->next = h2;

newtail = h2;

h2 = h2->next;

}



again: while (-) ✓

{ if (h1->d <= h2->d) ✓ 3 < 4

{

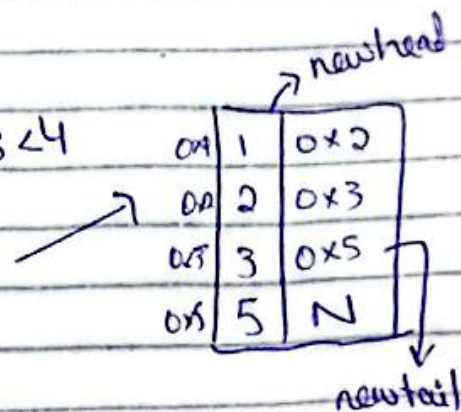
newtail->next = h1;

newtail = h1;

h1 = h1->next;

}

else { - }



again: while (-) ✓

{

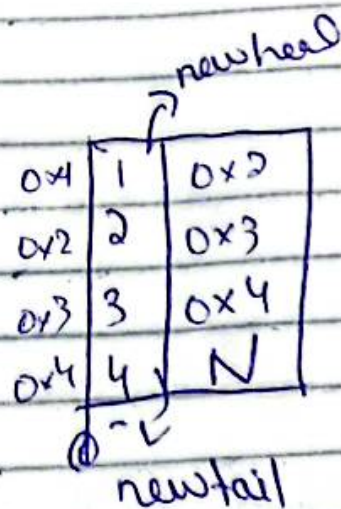
if (-) X 5 <= 4

else {

newtail->next = h2;

newtail = h2;

h2 = h2->next; }



while (-) X h2 = NULL

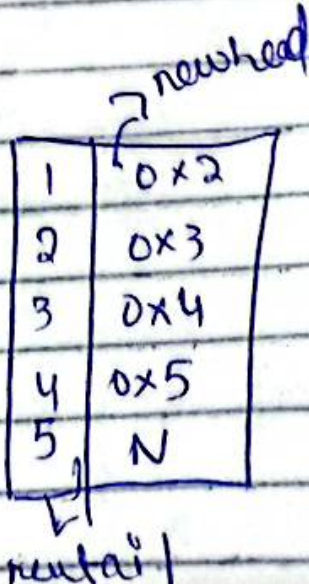
if (h2 == NULL) ✓

{

newtail->next = h1;

}

if (h2 != NULL) X



```
head = newhead;
tail = newtail;
```

// updated head & tail
(L2).

(Question # 03)

```
void findmiddle()
```

```
{
    Node* slow = head;
    Node* fast = head;
```

// point both to head
traversal

```
while (fast != N && fast->next != N)
{
```

```
    slow = slow->next;
```

// iterate 1 step

```
    fast = fast->next->next;
```

// iterate 2 steps

```
}
```

(DRY RUN)

Suppose: our list is

fast = 0x1

slow = 0x1

0x1	1	0x2
0x2	2	0x3
0x3	3	0x4
0x4	4	0x5
0x5	5	N

head

tail

Now: while (fast != N && fast->next != N) ✓

```
    slow = slow->next;
```

```
    fast = fast->next->next;
```

slow = 0x2

fast = 0x3

while (→) ✓

again

```
    slow = slow->next;
```

```
    fast = fast->next->next;
```

s = 0x3

f = 0x5

while(1) { fast->next = NULL

data is at slow: 0x3 -> 3

(Question # 04)

void deletehead()

Node* fast, *slow = head;

for(int i=0; i<n; i++) // move fast to that
{ print if exist

if(fast == NULL) // if n reaches head

{ Node* del = head; // save head
head = head->next;
delete del; // delete
return;

while(fast->next != NULL)

{ fast = fast->next; // move towards the node
slow = slow->next; to be deleted

Node* del = slow->next; //

slow->next = slow->next->next; // move slow to del's next

if(slow->next == NULL) // if fast is deleted.
{ tail = slow; } // if slow reaches end

delete del;

(DRY RUN)

Suppose, our list is

fast = 0x1
slow = 0x1

0x1	1	0x2
0x2	2	0x3
0x3	3	0x4
0x4	4	N

Let user enter 2:

1) for (int i=0; i<n; i++) ✓
{

if (fast == NULL) X

} fast = fast->next; f = 0x2

2) i=1, i<2; ✓

if (—) X

fast = fast->next; f = 0x3

3) i=2, i<2 X

4) if (fast == NULL) X

Now while (fast->next != NULL) ✓
{

fast = fast->next;

f = 0x4

slow = slow->next;

s = 0x2

}

Again while (—) X f->next = NULL

Node* del = slow->next; del -> 0x3

and

ROC

2nd last 3

(Question # 05)

```
void createcycle(int pos)
```

```
{  
    if (pos < 0) // if negative val  
    { return; }
```

```
    Node* temp = head;
```

```
    // to traverse
```

```
    Node* cycnode = NULL; // to find node for cycl
```

```
    int count = 0;
```

```
    while (temp != NULL) // run loop until temp = NULL
```

```
    {  
        if (count == pos) // if pos found
```

```
        {  
            cycnode = temp; // save that node
```

```
        }  
        if (temp->next == NULL) // if last node
```

```
        {  
            break;
```

```
        }  
        temp = temp->next; // increment node.  
        count++; // increment count
```

```
    }
```

```
    if (cycnode != NULL && temp != NULL)
```

```
    {  
        temp->next = cycnode;
```

```
        // point next of tail  
        // to that node
```

```
    }
```

```
bool hascycle()
```

```
{
    Node* slow = head;
    Node* fast = head;
```

// for traversing

```
while (fast != NULL && fast->next != NULL)
{
```

```
    slow = slow->next; // i+1;
    fast = fast->next->next; // i+2;
```

```
    if (slow == fast)
    {
```

```
        return true;
```

```
    }
```

```
}
```

```
return false;
```

```
}
```

// mean if tail->next
then fast will be
a loop & it
will never be null

if yes
then return

otherwise false.

(DRY RUN)

0	1	2	3
0x1	0x2	0x3	0x4

Suppose: Input list is

1 → 2 → 3 → 4

createcycle();

suppose pos = 2

temp = 1 (0x1)

```
a) while (temp != NULL) ✓
{
```

```
    if (count == pos) X 0 ≠ 2
```

```
    if (t->next == NULL) X
```

```
    temp = t->next;
```

```
    c++;
```

```
}
```

t = 0x2

c = 1

while() ✓

```
if (c == pos) 1 + 2 X
if (t == N) X
t = t -> next;
c++;
```

t = 0x3
c = 2

while() ✓

```
if (c == pos) 0 + 2 ✓
{
    cycnode = temp;
}
if (t == next == NU) X
temp = t -> next;
c++;
```

cycnode = 0x3

t = 0x4

while() ✓

```
if ( ) X
if (t == N) ✓
{ break; }
```

~~cycle created~~

if (cycnode == N && t == N) ✓

```
temp -> next = cycnode;
```

0x1	1	0x2
0x2	2	0x3
0x3	3	0x4
0x4	4	0x3

cycle created.

Question # 06

```
Node* gethead() // to access head
{
    return head;
}
```

```
SSL addition(SSL & l1, SSL & l2) // 2 arguments of:
{
    SSL result; // addition result
```

```
Node* h1 = l1.gethead(); // assign heads of
Node* h2 = l2.gethead(); // respectively
```

```
int carry = 0;
```

```
while(h1 != N || h2 != N || carry != 0) if all wrong - exit
```

```
{
    int sum = carry; // move prev carry to sum
```

```
    if(h1 != N)
    {
```

```
        sum += h1->data;
```

```
        h1 = h1->next;
```

```
    }
```

```
    if(h2 != N)
    {
```

```
        sum += h2->data;
```

```
        h2 = h2->next;
```

```
    }
```

```
    carry = sum / 10; // check if carry
```

1. Initialisation
 2. Traversal
 3. Deletion

1. Initialisation
 2. Traversal
 3. Deletion

1. Initialisation
 2. Traversal
 3. Deletion

1. Initialisation
 2. Traversal
 3. Deletion

1. Initialisation
 2. Traversal
 3. Deletion

1. Initialisation
 2. Traversal
 3. Deletion

again while(-)✓

{

int sum = 0;

s = 0

if(-)✓

{

s += hl->d;

s = 2

hl = hl->next;

hl = 0x3

}

if(-)✓

{

s += h2->d;

s = 7

h2 = h2->n;

h2 = 0x5

}

cout << sum/10;

c = 0

int dig = sum%10

74010 = 7

result.insertion(dig);

}

0x9	5	0x9
0x8	7	N

again while(-)✓

{

int s = 0;

s = 0

if(-)✓

{ s += hl->d;

3

hl = hl->n;

N

if(-)✓

{ s += h2->d;

9

h2 = h2->n;

N

cout << s/10;

c = 0

int d = s%10;

d = 9

res.ins(d);

return result;

}

0x9	5	0x9
0x8	7	0x9
↑ 0x9	9	N

Question # 07

```
Node* findmid()
```

```
{  
    Node* slow = head;  
    Node* fast = head;
```

```
    ... explained before // Q # 03
```

```
}
```

```
Node* reverse(Node* node)
```

```
{
```

```
    ... explained before // Q # 01
```

```
}
```

```
Node* void reorderlist()
```

```
{
```

```
    if (head == N || h->next == N) // 0 or 1 node  
    {  
        if yes → return
```

```
        return;
```

```
}
```

```
Node* mid = findmid();
```

```
// store mid into
```

```
Node* sechalf = reverse(mid);
```

```
// reverse after mid
```

```
Node* fhalf = head;
```

```
// store head into fhalf.
```

```
while (shalf->next != N) {
```

```
    Node* t1 = fh->next;
```

```
// cleared in DRY RUN
```

```
    Node* t2 = sh->next;
```

```
    fh->next = sh;
```

```
    sh->next = t1;
```

```
    fh = t1;
```

```
    sh = t2; }
```


sh → next = t2;

fh = t1; fh = 0x3
gh = t2; sh = 0x3

1	0x5
5	0x2
2	0x1
4	0x3
3	N

again { while (—) & (sh → next = N)

(Question # 08)

void removeduplicates();

if (head == NULL) { return; }

Node* curr = head;

while (curr != NULL && curr → next != NULL) // exit if last or 2nd last

{ if (curr → data == curr → next → data) // if same data

{ Node* duplicate = curr → next; // move 2nd same to
curr → next = curr → next → next; dup & // point curr
delete duplicate; to dup's next
// del that node.

else

{

curr = curr → next;

// if not same
iterate current.

}

}

DRY RUN

suppose: our list is 1 2 2 3 3

0x1 0x2 0x3 0x4 0x5

Node = head: cur = 0x1

Now: while (cur != N && cur->next != N) ✓

{
if (cur->d == cur->next->data) ✓ 1=1

{
Node* dup = cur->next; dup = 0x2
cur->next = cur->n->next; 0x1 -> 0x3
} delete dup;

else X } List Now

again: while (✓)

1 2 2 3
↓ ↓ ↓ ↓
0x1 0x2 0x4 0x5

if (✓) X 1 ≠ 2

else
{

c = c->n;

c = 0x3

}

again: while (✓)

{
if (✓) 2=2

dup = c->n;

dup = 0x4

c->n = c->n->n;

0x3 -> 0x5

} del dup;

else X

List now 1 2 3

again: while (✓)

if (✓) X 2 ≠ 3

else

c = c->n;

c = 3

again
while (✓)
c->n =

Question # 09

void reverse()

checking edge case & calling helper function

void reverseHelper(Node* curr, int k)
 // curr is head
 // k is node count to reverse

if (c == N || k <= 1)

// Again checking edge cases for recursion

return curr;

Node* temp = curr;

int count = 0;

while (t != N && c < k) // to check if we have k nodes to reverse

t = t->next;

c++;

if (c < k) { return curr; }

Node* prev = NULL;

Node* current = curr;

Node* next = NULL;

count = 0;

while (—)

{ Q # 01 }

if (next != NULL)

recursively calling if still nodes left

curr->next = reverseHelper(next, k);

return prev; // new head

(DRY RUN)

Suppose: Our list is
k=2

0x1 0x2 0x3 0x4 0x5
1 2 3 4 5

current = 0

// check if we have nodes to reverse k times (groups)

t = 0x1, curr = 0x1

1) while (t != N & c < k) ✓
{

t = t -> next

t = 0x2

c++;

c = 1

}

2) while (✓) ✓
{

t = t -> n;

t = 0x3

c++;

c = 2

3) while (✓) ✓ {

t = t -> n;

t = 0x4

c++;

c = 3

4) while (✓) ✓ {

t = t -> n;

0x5

c++;

c = 4

5) while (✓) ✓ {

t = t -> n;

N

c++;

c = 5

6) while (✓) ✗ [t = N]

Reverse DRY RUN -> Q1

if (next != NULL) then 0x3 then 0x5 then exit for at while (counting nodes)

{ reversing; } as it will be last node